

AMiT
Arba Minch Institute of Technology
አርባ ምንጭ ቴክኖሎጂ ኢንስቲትዩት
Arba Minch University
አርባ ምንጭ ዩኒቨርሲቲ

Faculty of Computing and Software Engineering
Department of Software Engineering

SENG5232- Software Architecture and Design

Chapter 2: Design Patterns

*"The best designers will use many **design patterns** that dovetail and intertwine to produce a greater whole."* *Erich Gamma*

By Aliazar D.(MSc in Software Eng.) Email :- aliazar.deneke@amu.edu.et

Contents

†**1** Pattern Oriented Design

†**2** Pattern Types

†**3** OO Design pattern

Pattern Oriented Design/#1

- Software architecture and design can be accomplished via two ways:

1. ***From scratch*** architecture and design: where the designer decomposes the system and compose the components all by him/her own.

2. ***Pattern oriented*** architecture and design: designer uses a proven model of system decomposition and component composition from previous experience of similar system design.

    - ***Pattern for software architecture*** describes a particular *recurring design problem* that arises in specific design contexts, and presents a well-proven generic scheme for *its solution*.

    - They typically shows relationships and interactions between classes or objects.

Pattern Oriented Design/#1

- The *solution scheme* is specified by describing its *constituent components*, their *responsibilities* and *relationships*, and the ways in which they collaborate.
- An established (ideal and proven) way of solving a problem with similar specific problem is known as a ***pattern for that problem***.
- A pattern addresses a *recurring design problem* that arises in specific *design situations*, and presents a solution to it.
- Pattern is described by triplets <*context, problem and solution*> .

Why Patterns?

Because:

1. *They document existing, well-proven design experience.*
2. *They identify and specify abstractions that are above the level of single classes and instances, or of components.*
3. *They provide a common vocabulary and understanding for design principles.*
4. *They are a means of documenting software architectures.*
5. *They support the construction of software with defined properties.*
6. *They help you build complex and heterogeneous software architectures .*
7. *They help you to manage so more complexity.*

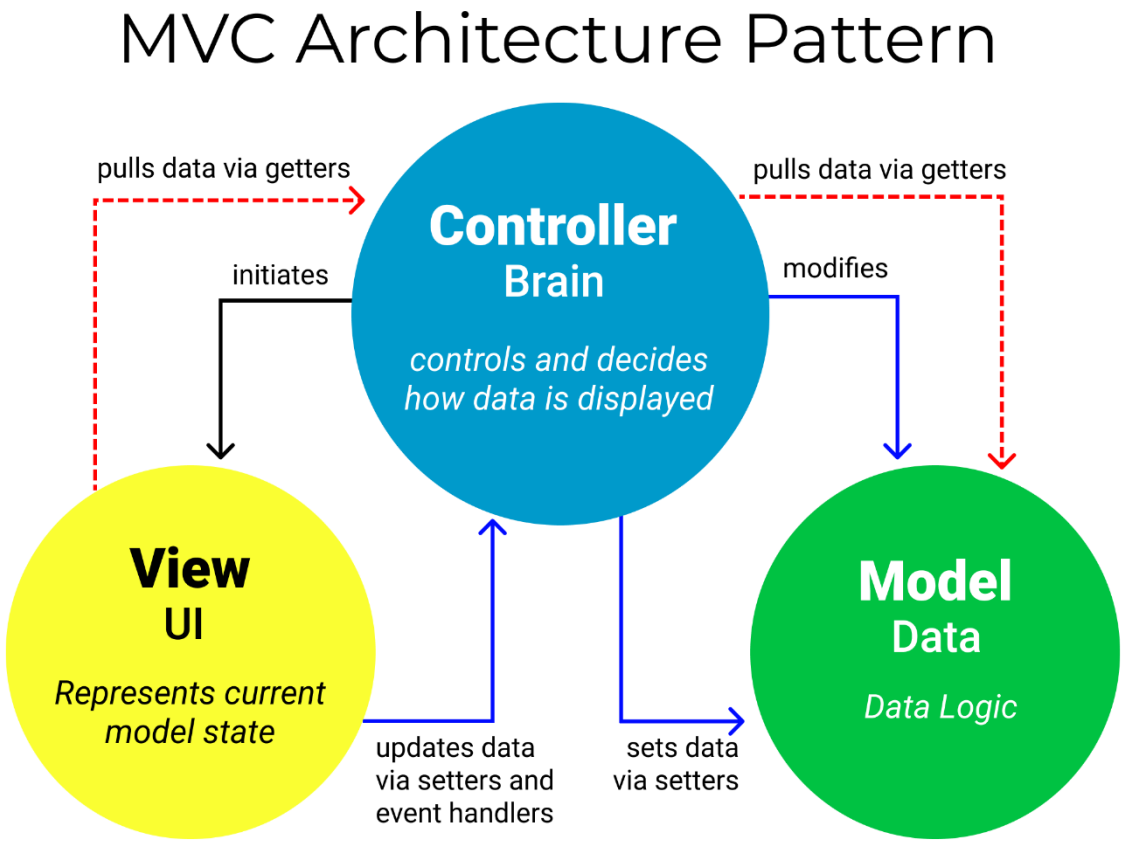
Are and aren't of patterns

- Patterns are means to *decompose the high level architecture* during AD and the *Components during detailed design* using AP and DP respectively.
- However, although a pattern *determines the basic structure of the solution* to a particular design problem, it does not specify a fully detailed solution.
- A pattern provides a *scheme for a generic solution* to a family of problems, rather than a prefabricated module that can be used 'as is'.

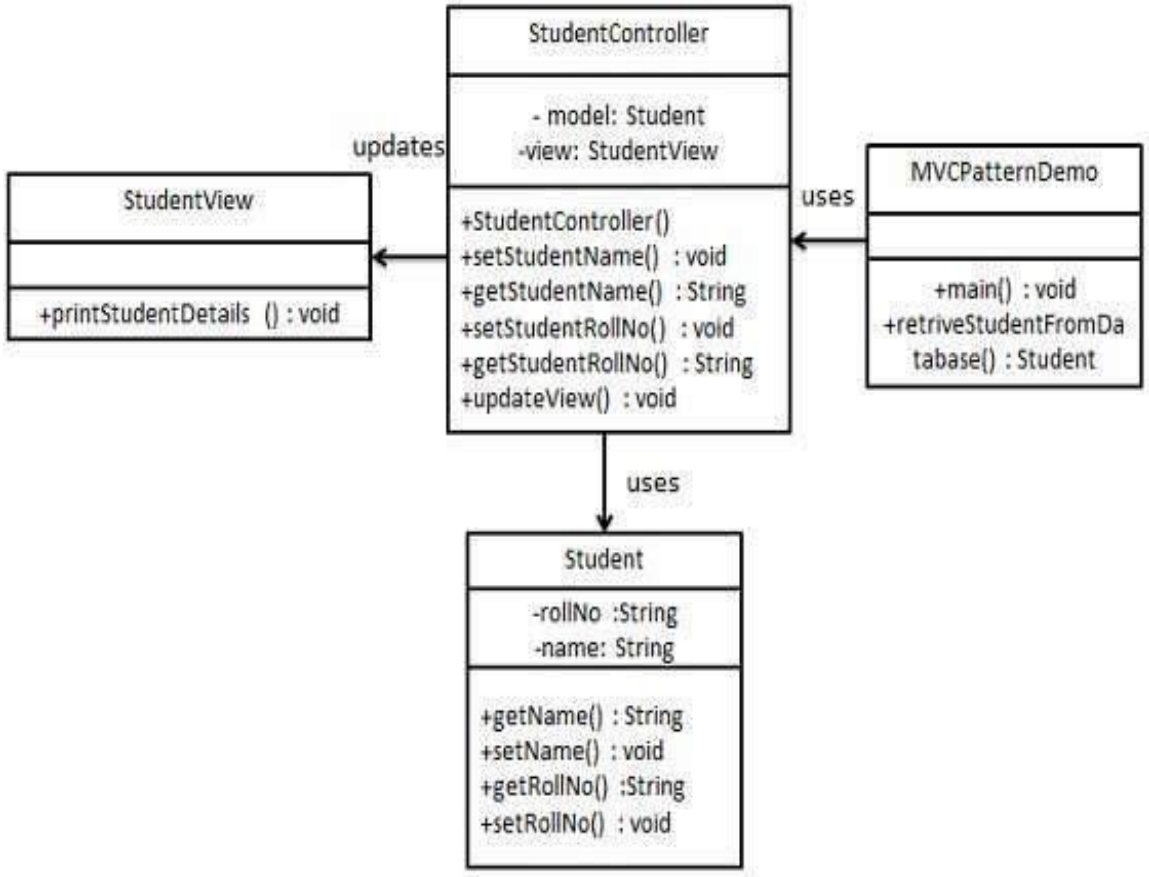
Are and aren't of patterns

- A pattern helps with the ***creation of similar units***. These units can be alike in their broad structure, but are frequently quite different in their detailed appearance.
    - *For Example: hospital management system, student information system and online library can be implemented using the MVC pattern as all are interactive application types. However, what is the Model, the View and the Controller in each of this system unless skeletally contently never be the same.*
- Patterns help solve problems, but they *do not provide complete solutions*.

Are and aren't of patterns



Student Management System



What constitutes a pattern?

» Pattern is described using a pattern description (schema) that specifies:

  » ***Context:*** *a situation giving rise to a problem.*

  » ***Problem:*** the recurring problem arising in that context.

  » ***Solution:*** a proven resolution of the problem.

» A *pattern schema* contains other details besides these to further clarify the pattern for selection including name, implementation, aka, known uses and etc.

Pattern description schema

Name	The name and a short summary of the pattern.
Also Known As	Other names for the pattern, if any are known.
Example	A real-world example demonstrating the existence of the problem and the need for the pattern.
Context	The situations in which the pattern may apply
Problem	The problem the pattern addresses, including a discussion of its associated forces.
Solution	The fundamental solution principle underlying the pattern.
Structure	A detailed specification of the structural aspects of the pattern, including CRC-cards for each participating component and an OMT class diagram.
Dynamics	Dynamical scenarios describing the run-time behavior of the pattern.
Implementation	Guidelines for implementing the pattern.
Variants	A brief description of variants or specializations of a pattern.
Known-uses	Examples of the use of the pattern, taken from existing systems.
Consequences	The benefits the pattern provides, and any potential liabilities.

Pattern Types/#2

- Based on their scope of influence in the system and usage during design phases, patterns are broadly classified as:

1. ***Architectural patterns which*** represent the highest-level patterns in our pattern system. They help you to specify the fundamental structure of an application. Each architectural pattern helps you to achieve a specific global system property, such as the ***adaptability of the user interface***.
2. ***Design Patterns***: those used to design a particular component in the system after decomposition of the system is done.
3. **Idioms**: are programming language level (implementation) patterns for recurring implementation problem.

Pattern Types/#2

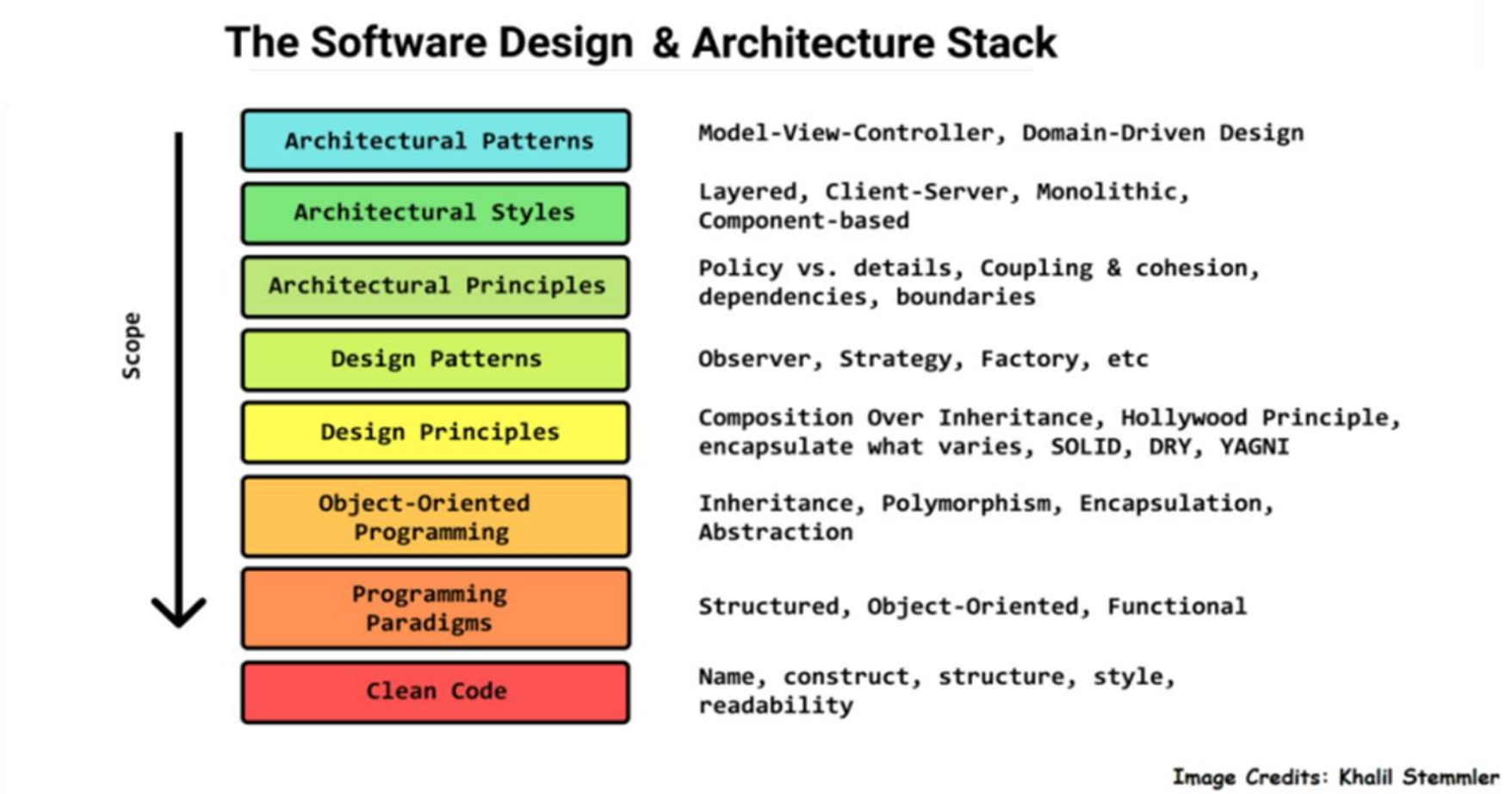
Known APs

- Layered AP
- Model View Controller AP
- Pipe and Filter AP
- Data Centric (Black-board) AP
- Other APs

Known DPs

- Adapter
- Composite
- Template
- Abstract Factory
- Observer
- Master-Slave
- Other DPs

Pattern Oriented Design



OO Design Patterns/#3

- What are design patterns
- Creational design patterns
- Structural design patterns
- Behavioral design patterns

" Design Patterns - Elements of Reusable Object-Oriented Software. " -

Erich Gamma

What are design patterns?

- They provides a general reusable solution for the common problems that occur in software design.
- They are programming language independent strategies for solving a common problem.
  - » That means a it represents an **idea**, not a particular implementation.
  - » The **idea** is to speed up the development process by providing *well-tested, proven development/design* paradigms.
- By using design patterns, we can make our *code more flexible, reusable, and maintainable*.
- It's *not mandatory* to always implement design patterns in your project.
- To find out which pattern to use, you just have to *try to understand the design patterns and their purposes.*

Example on design patterns

Example:

1. In many real-world situations, we want to create only one instance of a class.
    - For example, there can be only one active president of a country at any given time.
2. Other software examples could be a single DB connection shared by multiple objects as creating a separate DB connection for every object is costly.
    - Similarly, there can be a single configuration manager or error manager in an application that handles all problems instead of creating multiple managers.
3. Suppose we have an Object that loads data from database. Now we need to modify this data in our program multiple times, so it's not a good idea to create the Object using new keyword and load all the data again from database. So it clone the existing object into a new object and then do the data manipulation.

Types of Design Patterns

There are mainly three types of design patterns:

1. **Creational design patterns :-** These patterns deal with object creation mechanisms. They provide ways to create objects while hiding the details of how objects are instantiated.
    - **Examples** include the Singleton, Factory Method, and Abstract Factory patterns.
2. **Structural design patterns:** These patterns are concerned with how classes and objects are composed to form larger structures. They help define relationships between objects and provide flexibility in composing them.
    - **Examples** include the Adapter, Bridge, and Composite patterns.
3. **Behavioral design patterns:-** These patterns focus on communication between objects and responsibilities of objects. They describe how objects interact and communicate with each other to accomplish specific tasks.
    - **Examples** include the Observer, Strategy, and Command patterns.

References

*In 1990 a group called the **Gang Of Four** or "GOF" (gamma, helm, johnson, vlissides) compiled a catalog of design patterns in 1995 :The title of their book was "Elements of reusable object-oriented software".*

GOF Design Patterns

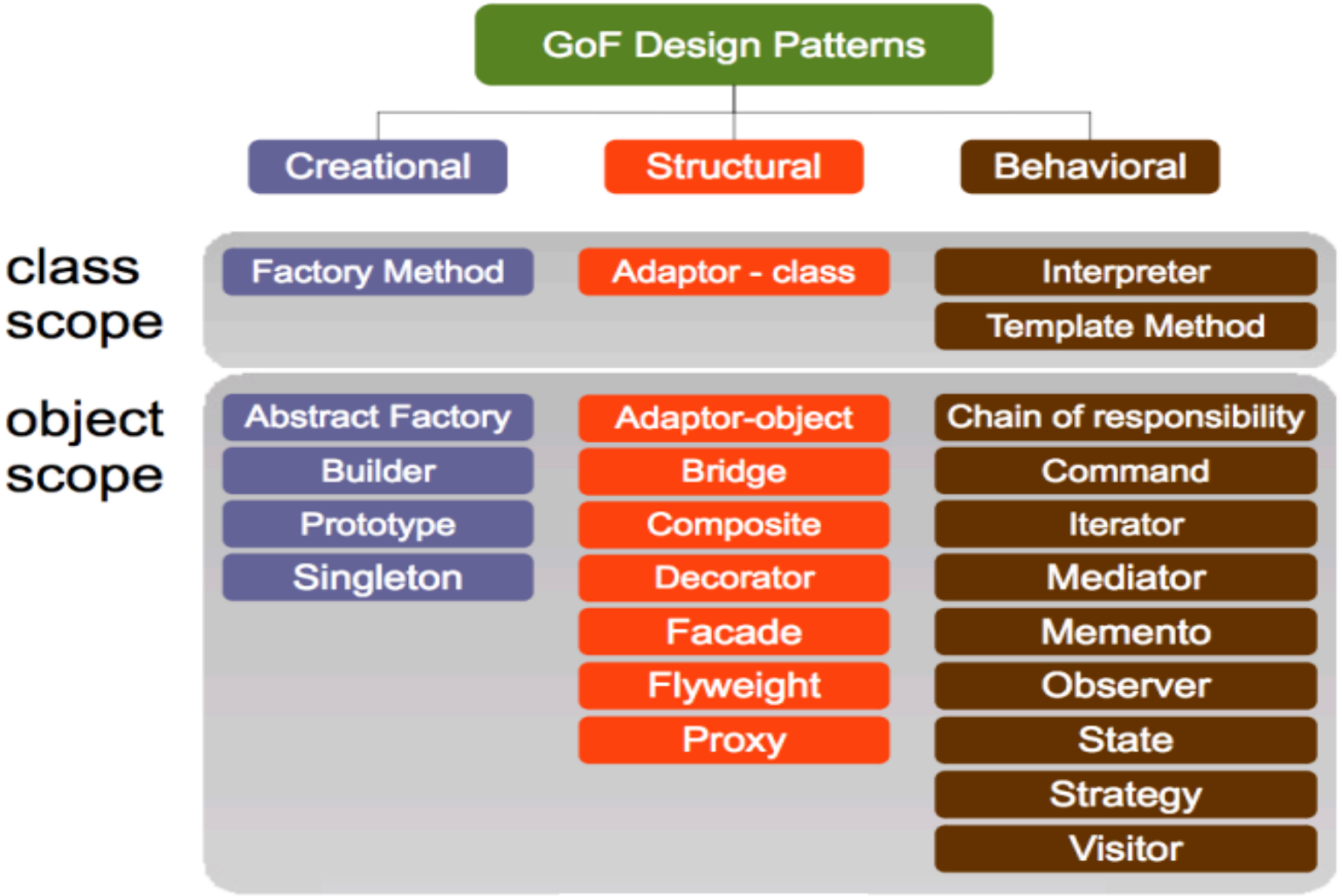


Classified by two criteria

1. **Purpose**:
   - » Creational : Concerns the creation process of objects and classes.
   - » Structural : Concerns the composition of classes and objects.
   - » Behavioral : Characterizes the interaction and responsibility of objects and classes.
2. **Scope**:
   - » Class scope: relationship between classes and subclasses, defined statically.
   - » Object scope: object relationships, dynamic.

GOF Design Patterns





What are Creational Design Patterns (CDPs)?

- CDPs deals with object creation mechanisms, trying to create objects in a manner suitable to the situation and aims to separate a system from how its objects are created, composed, and represented.
- CDPs provide various ways to create objects while abstracting the process of how objects are instantiated and initialized.
- CDPs help manage the complexities involved in object creation, making it more flexible, efficient, and suitable for different scenarios.
- CDPs are particularly useful when you need to ensure that a class has only one instance or when you want to decouple the system from the specific classes it creates.

….CDPs

- There are two recurring themes in these patterns.
  - First, they all encapsulate knowledge about which concrete classes the system uses.
  - Second, they hide how instances of these classes are created and put together.
  - All the system at large knows about the objects is their interfaces as defined by abstract classes.
- Consequently, the creational patterns give you a lot of flexibility in *what* gets created, *who* creates it, *how* it gets created, and *when*.
- They let you configure a system with "product" objects that vary widely in structure and functionality. Configuration can be static (that is, specified at compile-time) or dynamic (at run-time).

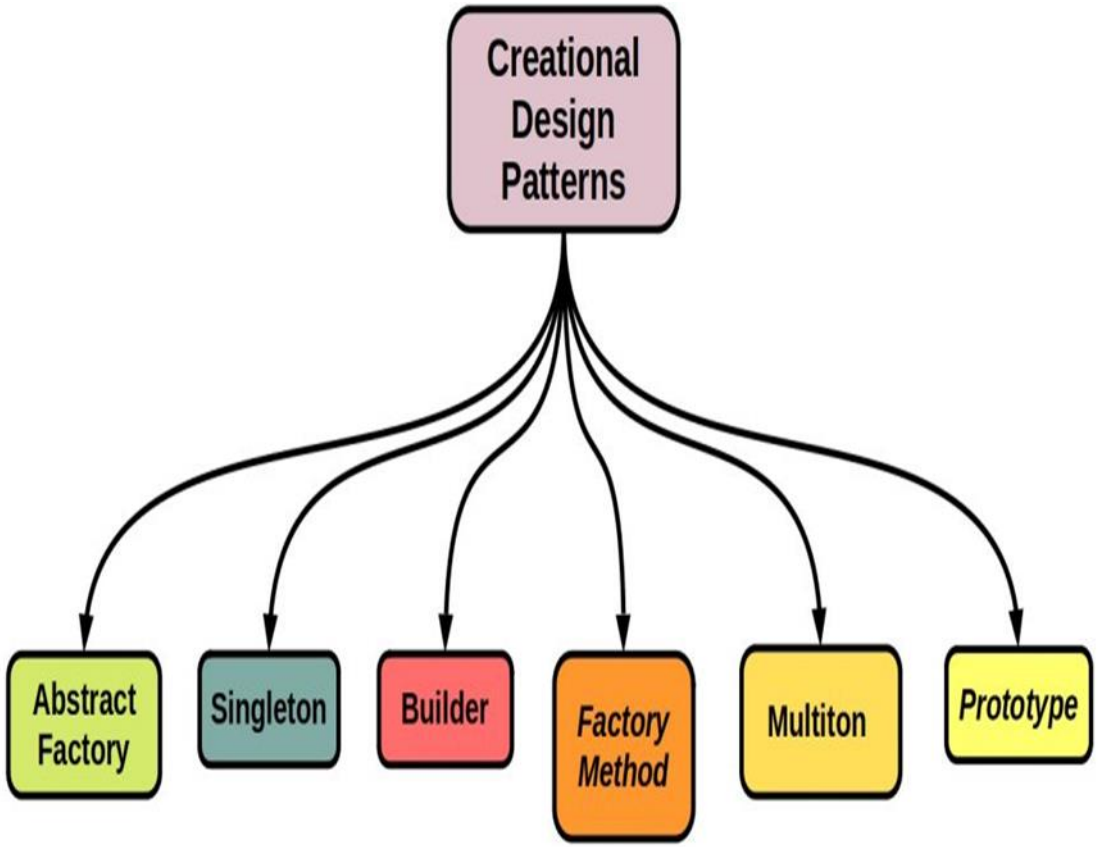
….CDPs

- They are all about class instantiation or object creation.
- They can be categorized into
  - **Class-Creational Pattern (CCP)**
    - It use inheritance effectively in the instantiation process.
  - **Object-Creational Patterns (OCP)**.
    - It use delegation effectively to get the job done i.e. the creation of objects is delegated to another component or class.

Why we use a creational design patterns?

- Some situations are *more complex* than simple instantiation can handle.
- Some situations have *complex consequences* if objects aren't instantiated in the right way or the right order.
- Some situations require that *only one object* is ever created.

Examples



Factory Design Pattern

- The Factory Pattern or Factory Method Pattern is also known as **Virtual Constructor**.
- It says that **just define an interface or abstract class for creating an object but let the subclasses decide which class to instantiate. In other words, subclasses are responsible to create the instance of the class.**
- **Advantage of Factory Design Pattern**
  - It allows the sub-classes to choose the type of objects to create.
  - It promotes the loose-coupling by eliminating the need to bind application-specific classes into the code. i.e. the code interacts solely with the resultant interface or abstract class, so that it will work with any classes that implement that interface or that extends that abstract class.

Important Points of FDP

- FDP provides approach to code for *interface rather than implementation*.
- FDP *removes the instantiation of actual implementation* classes from client code.
- FDP makes our *code more robust, less coupled and easy to extend*.
- FDP provides *abstraction* between implementation and client classes through inheritance.

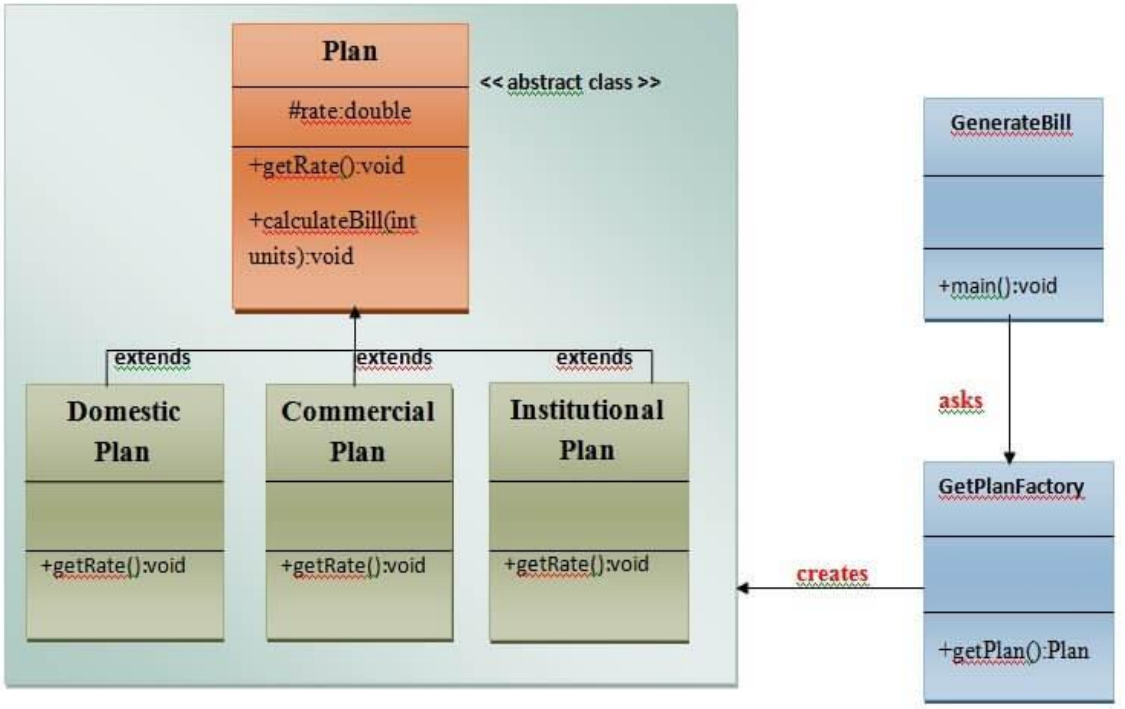
- **Usage of Factory Design Pattern**
  - When a class doesn't know what sub-classes will be required to create.
  - When a class wants that its sub-classes specify the objects to be created.
  - When the parent classes choose the creation of objects to its sub-classes.

…FDP How it works?

- **Client Code:** This is the code that needs an object but doesn't want to be tightly coupled with the specific class it's creating. Instead of creating objects directly, it calls a factory method or class to create the object for it.
- **Factory:** The factory is responsible for creating instances of objects. It may have one or more methods that take some parameters and decide which concrete class to instantiate based on those parameters. The client code interacts with the factory, not with the actual implementation classes.
- **Concrete Classes:** These are the actual implementation classes that the factory can create. The client code doesn't need to know about these classes, and they can change or be extended without affecting the client code.

Example

- We are going to create a **Plan** abstract class and concrete classes that extends the **Plan** abstract class.
- A factory class **GetPlanFactory** is defined as a next step.
- **GenerateBill** class will use **GetPlanFactory** to get a **Plan** object.
- It will pass information (**DOMESTICPLAN / COMMERCIALPLAN / INSTITUTIONALPLAN**) to **GetPalnFactory** to get the type of object it needs.



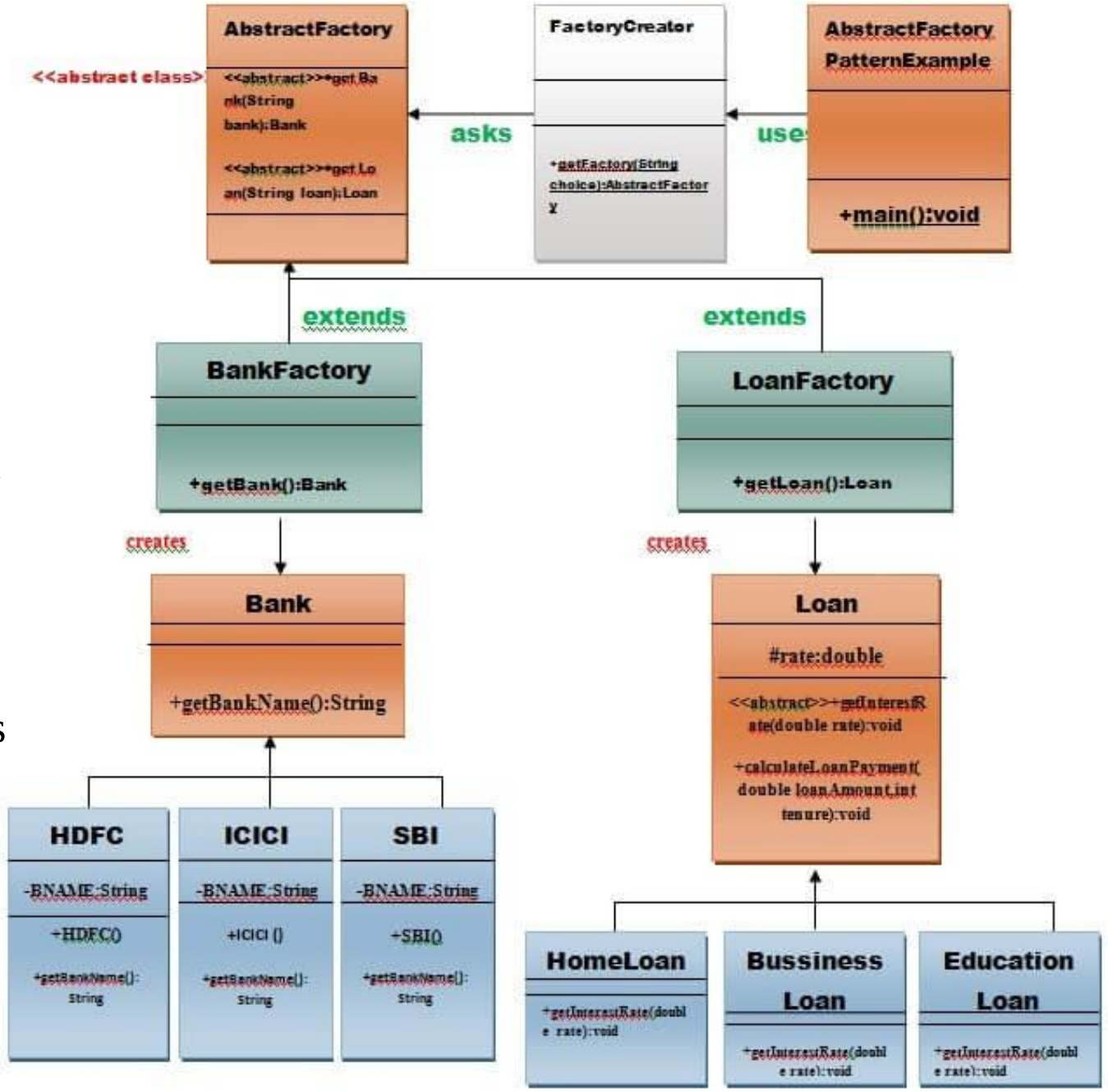
Abstract Factory Design Pattern

- It is almost similar to Factory Pattern except the fact that its more like ***factory of factories***.
- Abstract Factory Pattern says that **just define an interface or abstract class for creating families of related (or dependent) objects but without specifying their concrete sub-classes. That means it lets a class returns a factory of classes. So, this is the reason that it is one level higher than the Factory Pattern.**
- It is also known as **Kit**.
- **Advantage of Abstract Factory Pattern**
  - Abstract Factory Pattern isolates the client code from concrete (implementation) classes.
  - It eases the exchanging of object families.
  - It promotes consistency among objects.
- **Usage of Abstract Factory Pattern**
  - When the system needs to be independent of how its object are created, composed, and represented.
  - When the family of related objects has to be used together, then this constraint needs to be enforced.
  - When you want to provide a library of objects that does not show implementations and only reveals interfaces.
  - When the system needs to be configured with one of a multiple family of objects.

..AFDP

Example

- We are going to create a **Bank** interface and a **Loan** abstract class as well as their sub-classes.
- Then we will create **AbstractFactory** class as next step.
- Then after we will create concrete classes, **BankFactory**, and **LoanFactory** that will extends **AbstractFactory** class.
- After that, **AbstractFactoryPatternExample** class uses the **FactoryCreator** to get an object of **AbstractFactory** class.



Builder Pattern Design

- **Builder Pattern** says that **"construct a complex object from simple objects using step-by-step approach".**
- It is mostly used when object can't be created in single step like in the de-serialization of a complex object.
- It is introduced to solve *some of the problems* with Factory and Abstract Factory design patterns when the Object contains a lot of attributes.
- It solves the issue with a large number of optional parameters and inconsistent state by providing a way to build the object step-by-step and provide a method that will actually return the final Object.

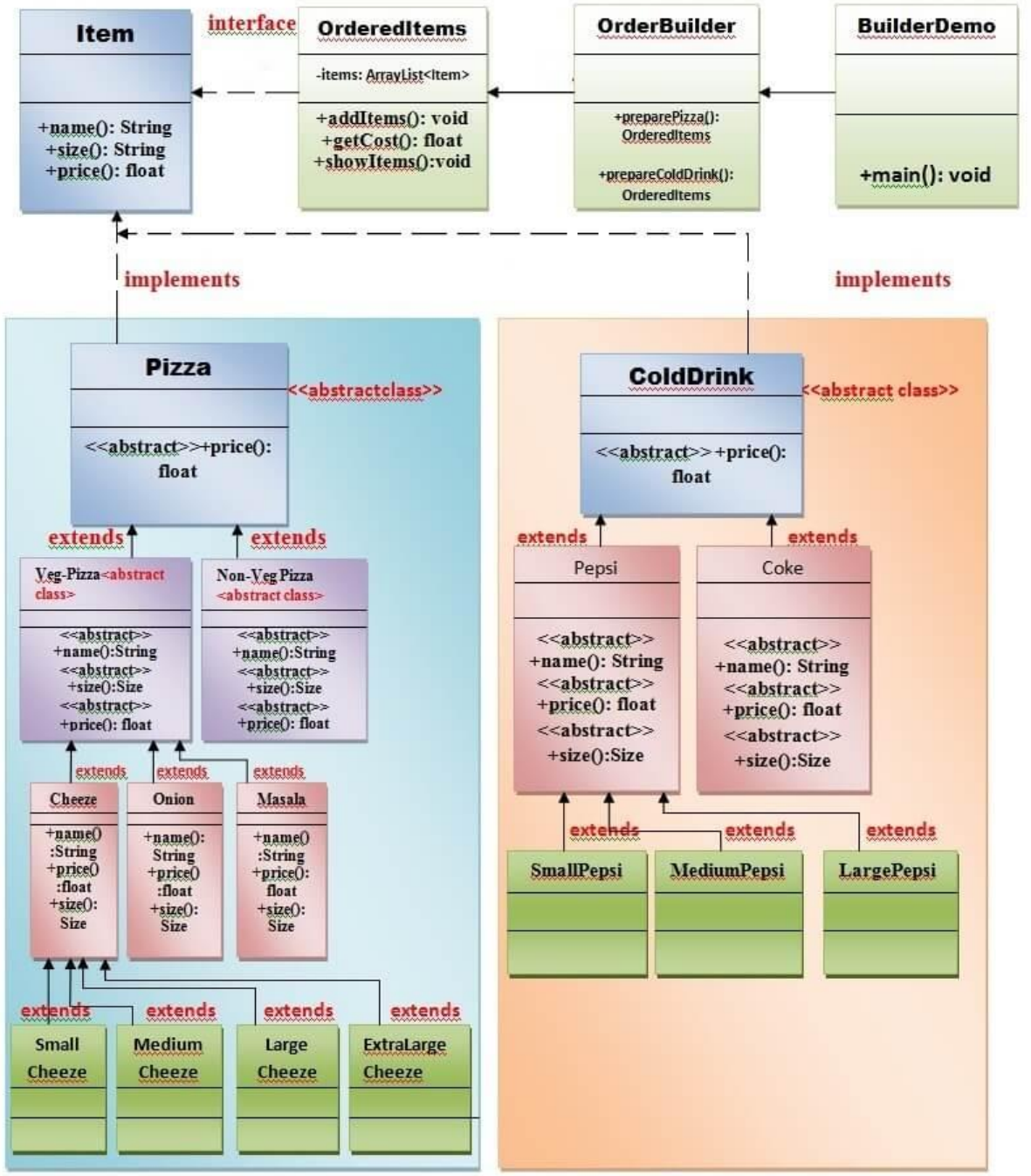
- **Advantage of Builder Design Pattern**
  - It provides clear separation between the construction and representation of an object.
  - It provides better control over construction process.
  - It supports to change the internal representation of objects.

...BPD

- We are considering a business case of **pizza-hut** where we can get different varieties of pizza and cold-drink.

- **Pizza** can be either a **Veg pizza** or **Non-Veg pizza** of several types (like **cheese pizza**, **onion pizza**, **masala-pizza** etc) and will be of 4 sizes i.e. **small**, **medium**, **large**, **extra-large**.

- **Cold-drink** can be of several types (like **Pepsi**, **Coke**, **Dew**, **Sprite**, **Fanta**, **Maaza**, **Limca**, **Thums-up** etc.) and will be of 3 sizes **small**, **medium**, **large**.



Singleton Pattern

- Singleton Pattern says that just "**define a class that has only one instance and provides a global point of access to it".**
- In other words, a class must ensure that only single instance should be created and single object can be used by all other classes.

- There are two forms of singleton design pattern
  - **Early Instantiation**: creation of instance at load time.
  - **Lazy Instantiation**: creation of instance when required.

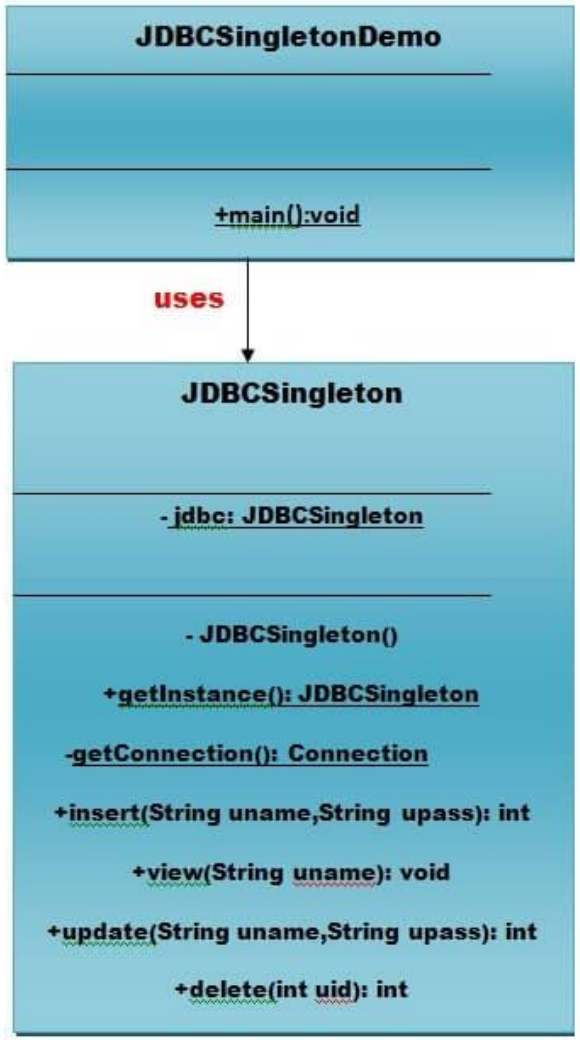
- **Advantage of Singleton Pattern**
  - Saves memory because object is not created at each request. Only single instance is reused again and again.

- **Usage of Singleton Pattern**
  - Singleton pattern is mostly used in multi-threaded and database applications. It is used in logging, caching, thread pools, configuration settings etc.

...SP

- **When to Use SP**
  - In multi-threading environments when multiple threads must access the same resources through the same singleton object.
  - When we must ensure that only one instance of a class is created.
  - When the instance must be available through all the code.

.



•We are going to create a **JDBCSingleton** class.

• This **JDBCSingleton** class contains its constructor as private and a private static instance jdbc of itself.

• **JDBCSingleton** class provides a static method to get its static instance to the outside world.

•Now, **JDBCSingletonDemo** class will use **JDBCSingleton** class to get the **JDBCSingleton** object.

Prototype Pattern

- Prototype Pattern says that **cloning of an existing object instead of creating new one and can also be customized as per the requirement.**
- This pattern should be followed, if the cost of creating a new object is expensive and resource intensive.

✓ **Advantage of Prototype Pattern**

• It reduces the need of sub-classing.

• It hides complexities of creating objects.

• The clients can get new objects without knowing which type of object it will be.

• It lets you add or remove objects at runtime.

✓ **Usage of Prototype Pattern**

• When the classes are instantiated at runtime.

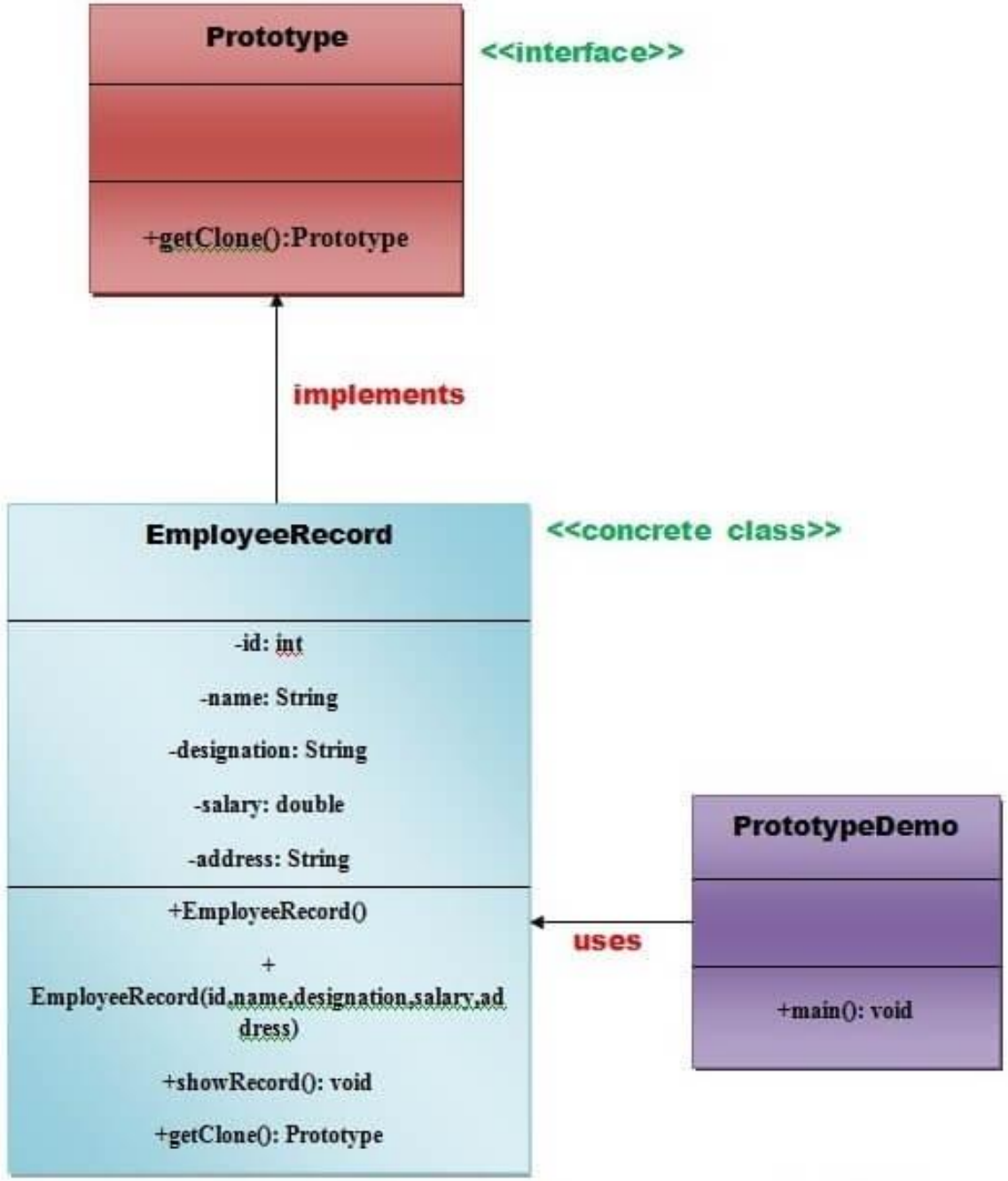
• When the cost of creating an object is expensive or complicated.

• When you want to keep the number of classes in an application minimum.

• When the client application needs to be unaware of object creation and representation.

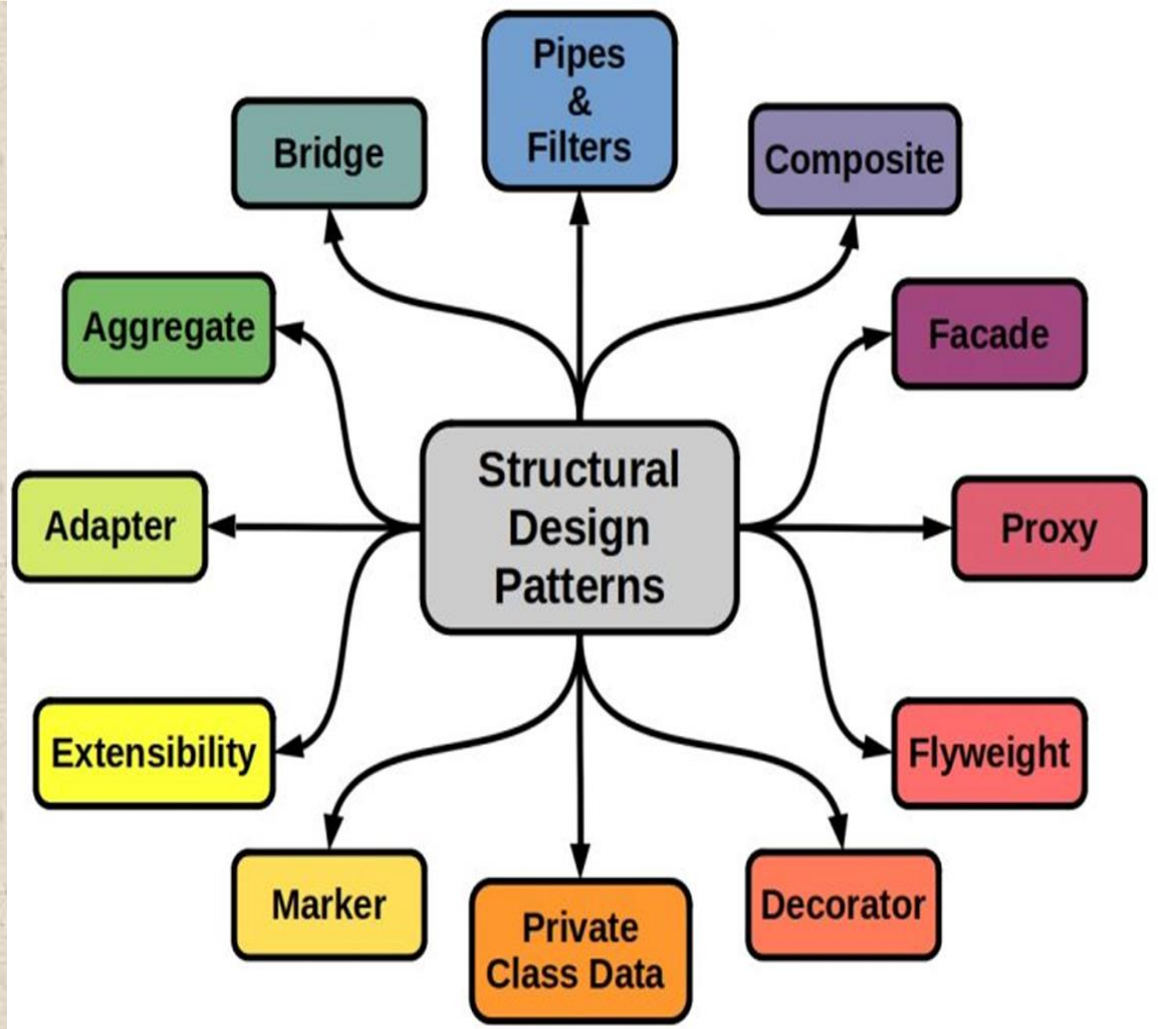
..PP

- We are going to create an interface **Prototype** that contains a method **getClone()** of **Prototype** type.
- Then, we create a concrete class **EmployeeRecord** which implements **Prototype** interface that does the cloning of **EmployeeRecord** object.
- **PrototypeDemo** class will uses this concrete class **EmployeeRecord**.



Structural Design Patterns

- **Structural design patterns** are concerned with how classes and objects can be composed, to form larger structures.
- The structural design patterns **simplifies the structure by identifying the relationships**.
- These patterns focus on, how the classes inherit from each other and how they are composed from other classes.



Adapter Pattern

- An **Adapter Pattern** says that just **"converts the interface of a class into another interface that a client wants".**
- In other words, to provide the interface according to client requirement while using the services of a class with a different interface.
- The **Adapter Pattern** is also known as **Wrapper**.

Advantage of Adapter Pattern

- It allows two or more previously incompatible objects to interact.
- It allows reusability of existing functionality.

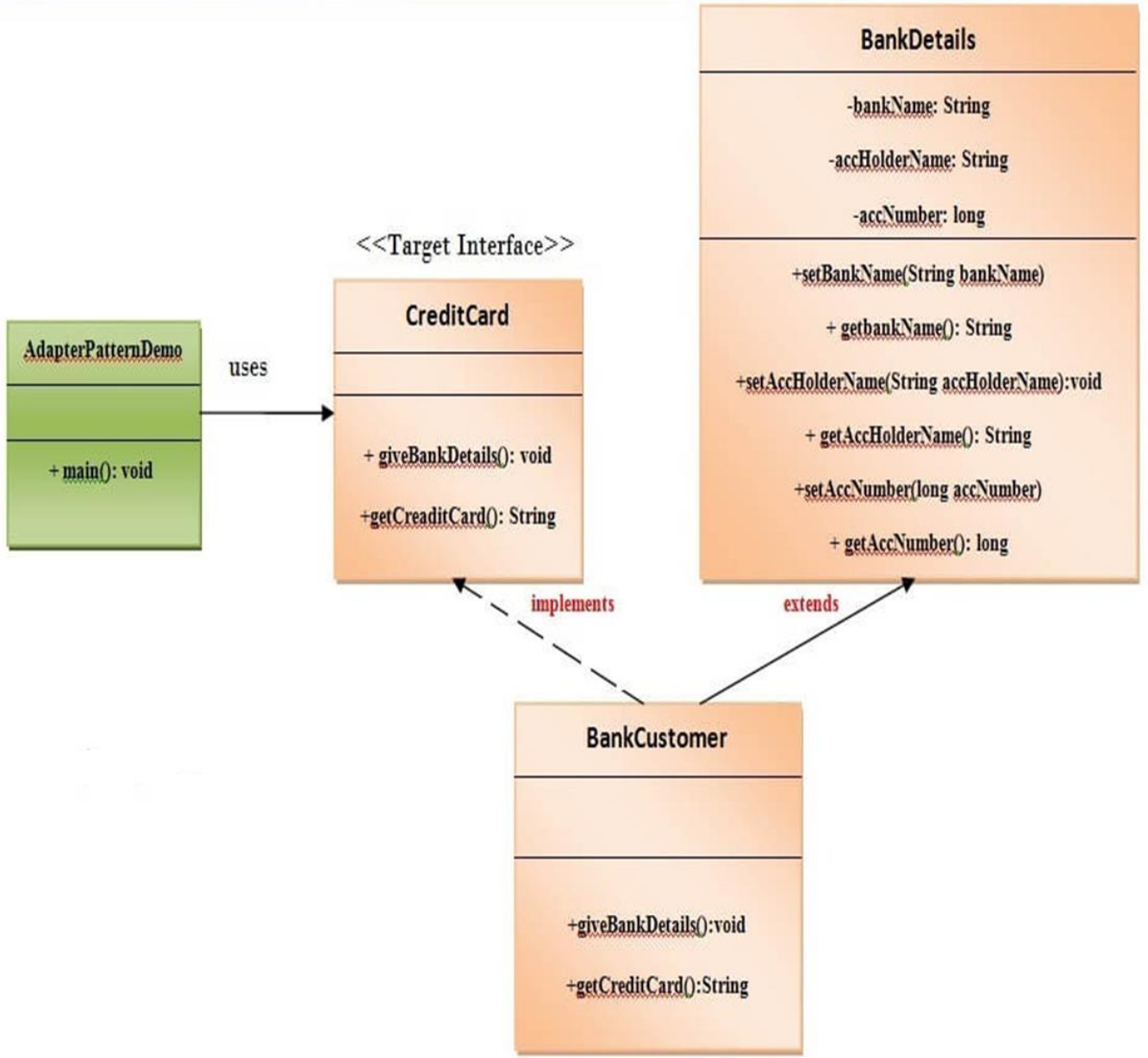
Usage of Adapter pattern

- When an object needs to utilize an existing class with an incompatible interface.
- When you want to create a reusable class that cooperates with classes which don't have compatible interfaces.
- When you want to create a reusable class that cooperates with classes which don't have compatible interfaces.

..AP

There are the following specifications for the adapter pattern:

- **Target Interface:** This is the desired interface class which will be used by the clients.
- **Adapter class:** This class is a wrapper class which implements the desired target interface and modifies the specific request available from the **Adaptee class**.
- **Adaptee class:** This is the class which is used by the **Adapter class** to reuse the existing functionality and modify them for desired use.
- **Client:** This class will interact with the **Adapter class**.



Bridge Pattern

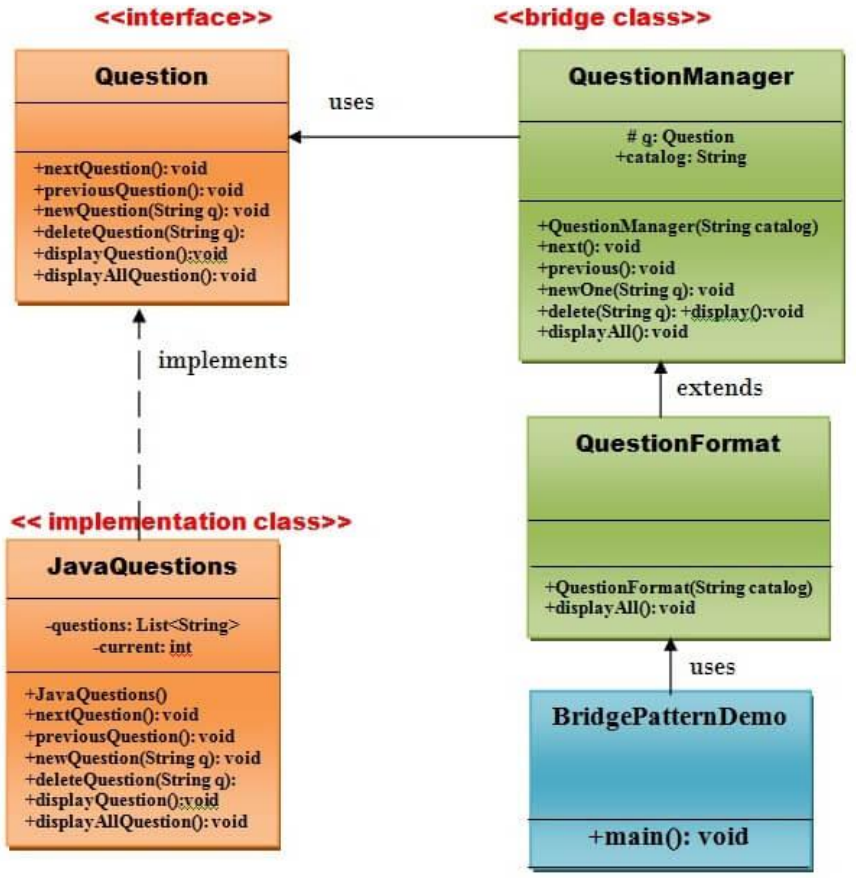
- A Bridge Pattern says that just **"decouple the functional abstraction from the implementation so that the two can vary independently".**
- The Bridge Pattern is also known as **Handle or Body.**

Advantage of Bridge Pattern

- It enables the separation of implementation from the interface.
- It improves the extensibility.
- It allows the hiding of implementation details from the client.

Usage of Bridge Pattern

- When you don't want a permanent binding between the functional abstraction and its implementation.
- When both the functional abstraction and its implementation need to extended using sub-classes.
- It is mostly used in those places where changes are made in the implementation does not affect the clients.



Composite Pattern

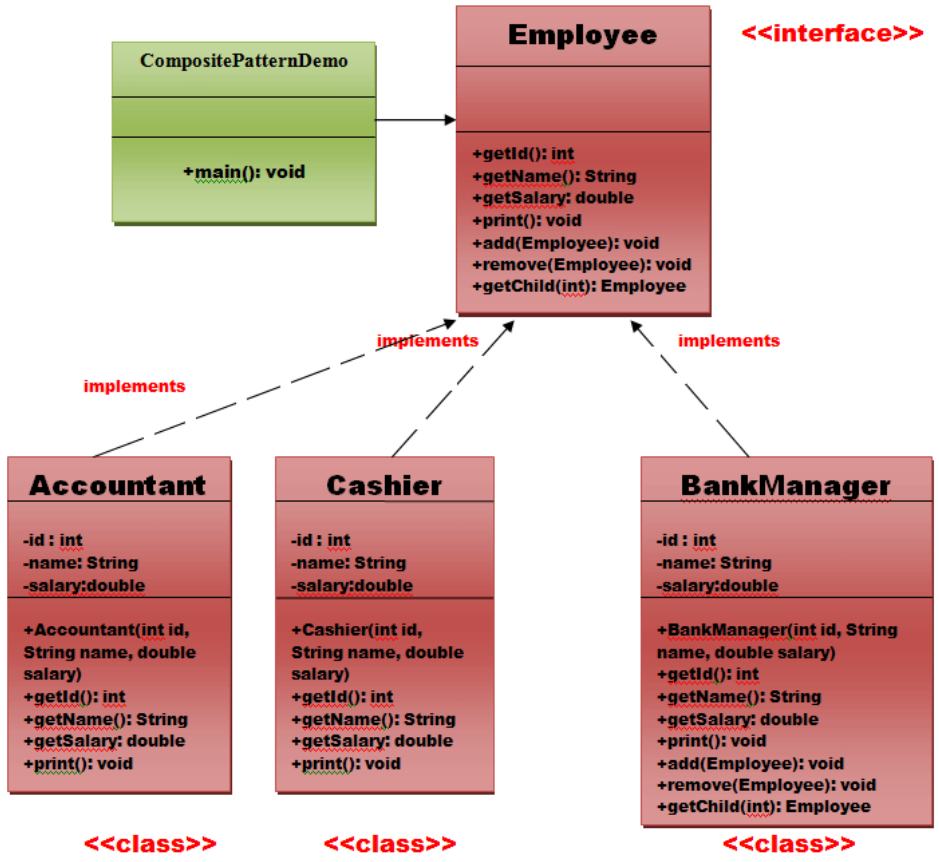
- A Composite Pattern says that just "**allow clients to operate in generic manner on objects that may or may not represent a hierarchy of objects".**

Advantage of Composite Design Pattern

- It defines class hierarchies that contain primitive and complex objects.
- It makes easier to you to add new kinds of components.
- It provides flexibility of structure with manageable class or interface.

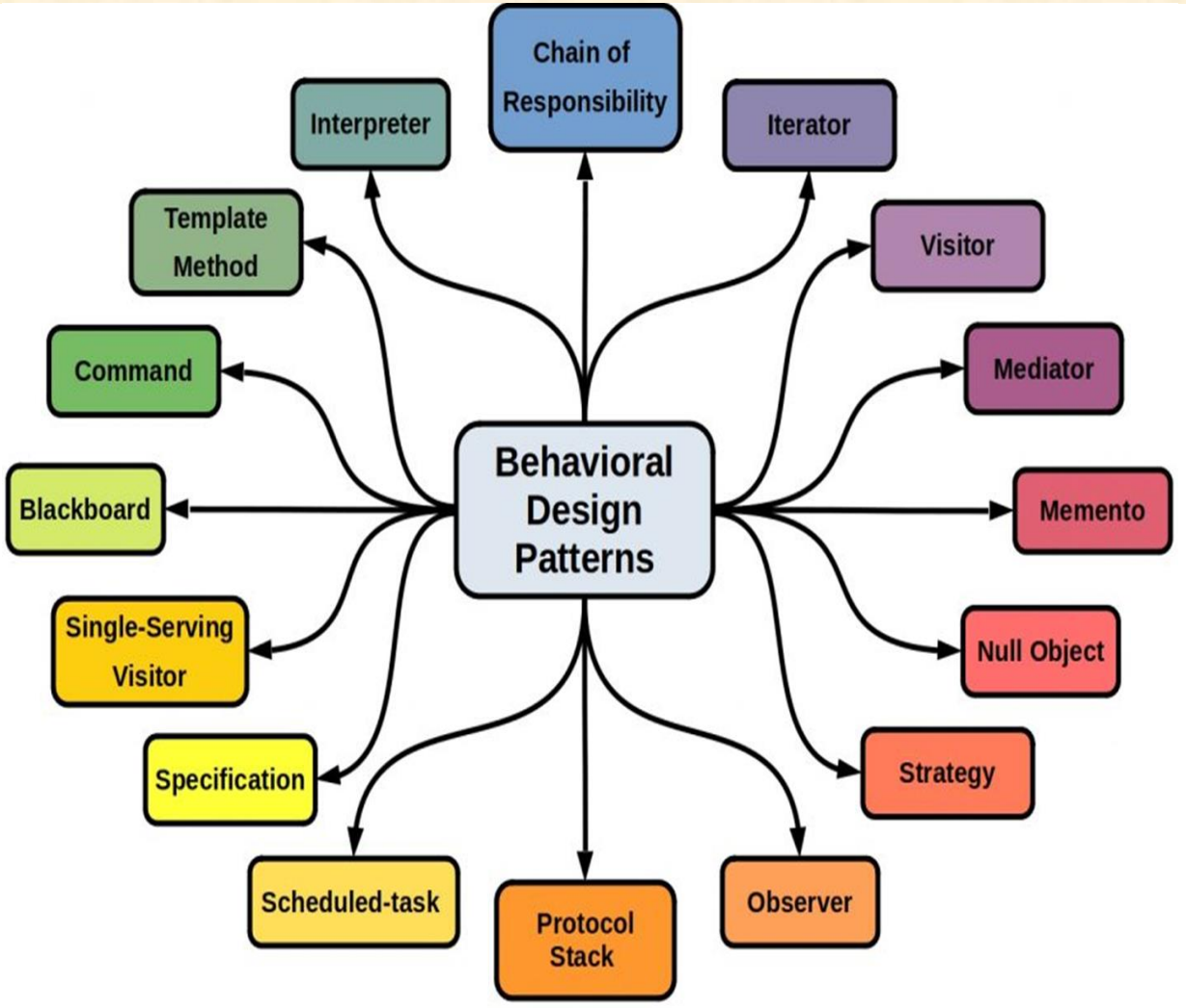
Usage of Composite Pattern

- When you want to represent a full or partial hierarchy of objects..
- When the responsibilities are needed to be added dynamically to the individual objects without affecting other objects. Where the responsibility of object may vary from time to time.



Behavioral Design Patterns

- Behavioral design patterns are concerned with the interaction and responsibility of objects.
- In these design patterns, the interaction between the objects should be in such a way that they can easily talk to each other and still should be loosely coupled.
- That means the implementation and the client should be loosely coupled in order to avoid hard coding and dependencies.



Chain Of Responsibility Pattern

- In chain of responsibility**, sender sends a request to a chain of objects. The request can be handled by any object in the chain.**
- A Chain of Responsibility Pattern says that just "**avoid coupling the sender of a request to its receiver by giving multiple objects a chance to handle the request"**. For example, an ATM uses the Chain of Responsibility design pattern in money giving process.
- In other words, we can say that normally each receiver contains reference of another receiver. If one object cannot handle the request then it passes the same to the next receiver and so on.

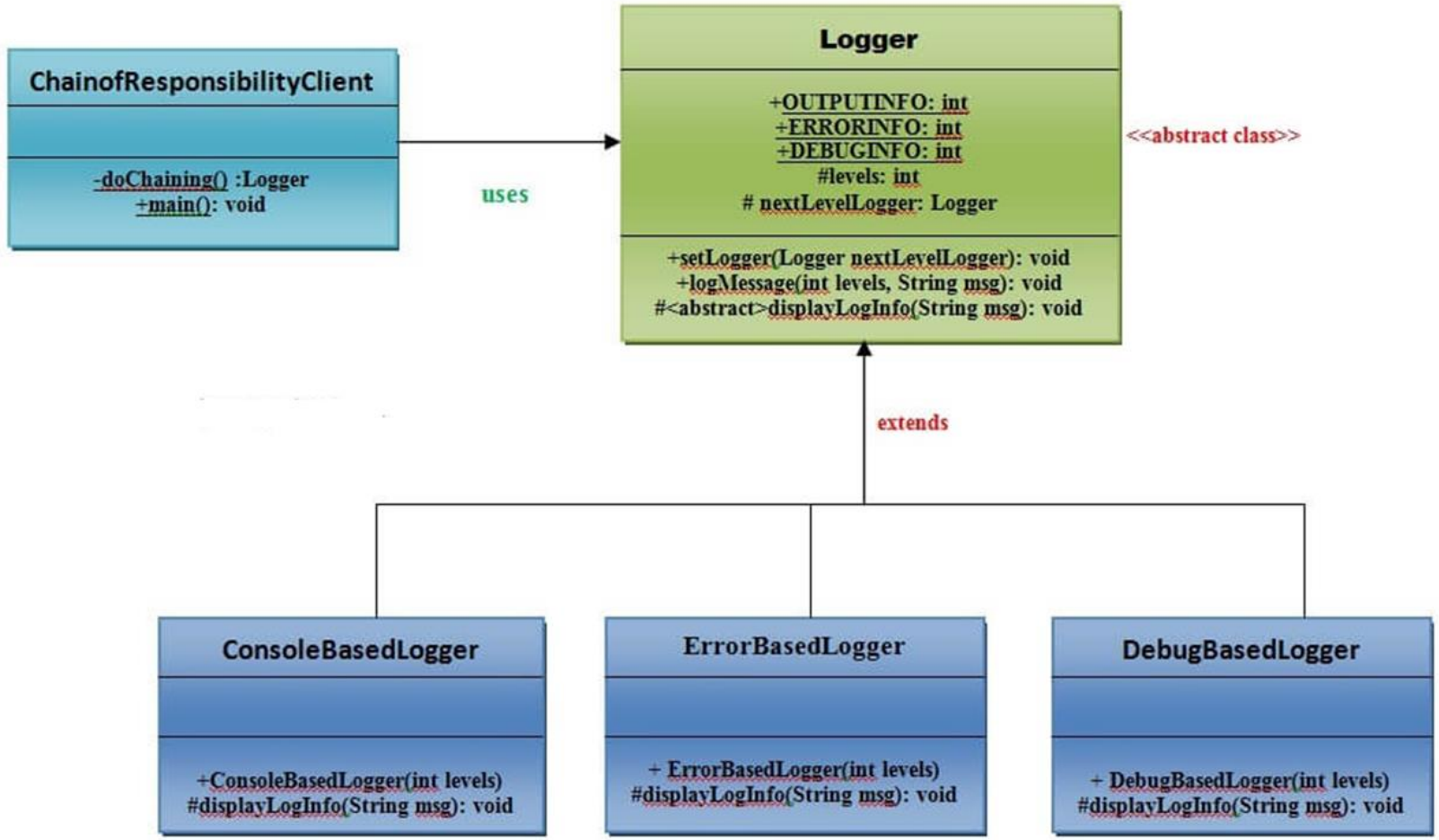
Advantage of Chain of Responsibility Pattern

- It reduces the coupling.
- It adds flexibility while assigning the responsibilities to objects.
- It allows a set of classes to act as one; events produced in one class can be sent to other handler classes with the help of composition.

Usage of Chain of Responsibility Pattern:

- When more than one object can handle a request and the handler is unknown.
- When the group of objects that can handle the request must be specified in dynamic way.

....CoRP


ChainofResponsibilityClient
-doChaining() :Logger
+main(): void
uses
Logger
+OUTPUTINFO: int
+ERRORINFO: int
+DEBUGINFO: int
#levels: int
nextLevelLogger: Logger
+setLogger(Logger nextLevelLogger): void
+logMessage(int levels, String msg): void
#<abstract>displayLogInfo(String msg): void
<<abstract class>>
extends
ConsoleBasedLogger
+ConsoleBasedLogger(int levels)
#displayLogInfo(String msg): void
ErrorBasedLogger
+ ErrorBasedLogger(int levels)
#displayLogInfo(String msg): void
DebugBasedLogger
+ DebugBasedLogger(int levels)
#displayLogInfo(String msg): void

Command Pattern

- A Command Pattern says that *"encapsulate a request under an object as a command and pass it to invoker object. Invoker object looks for the appropriate object which can handle this command and pass the command to the corresponding object and that object executes the command".*
- It is also known as **Action or Transaction.**

Advantage of command pattern

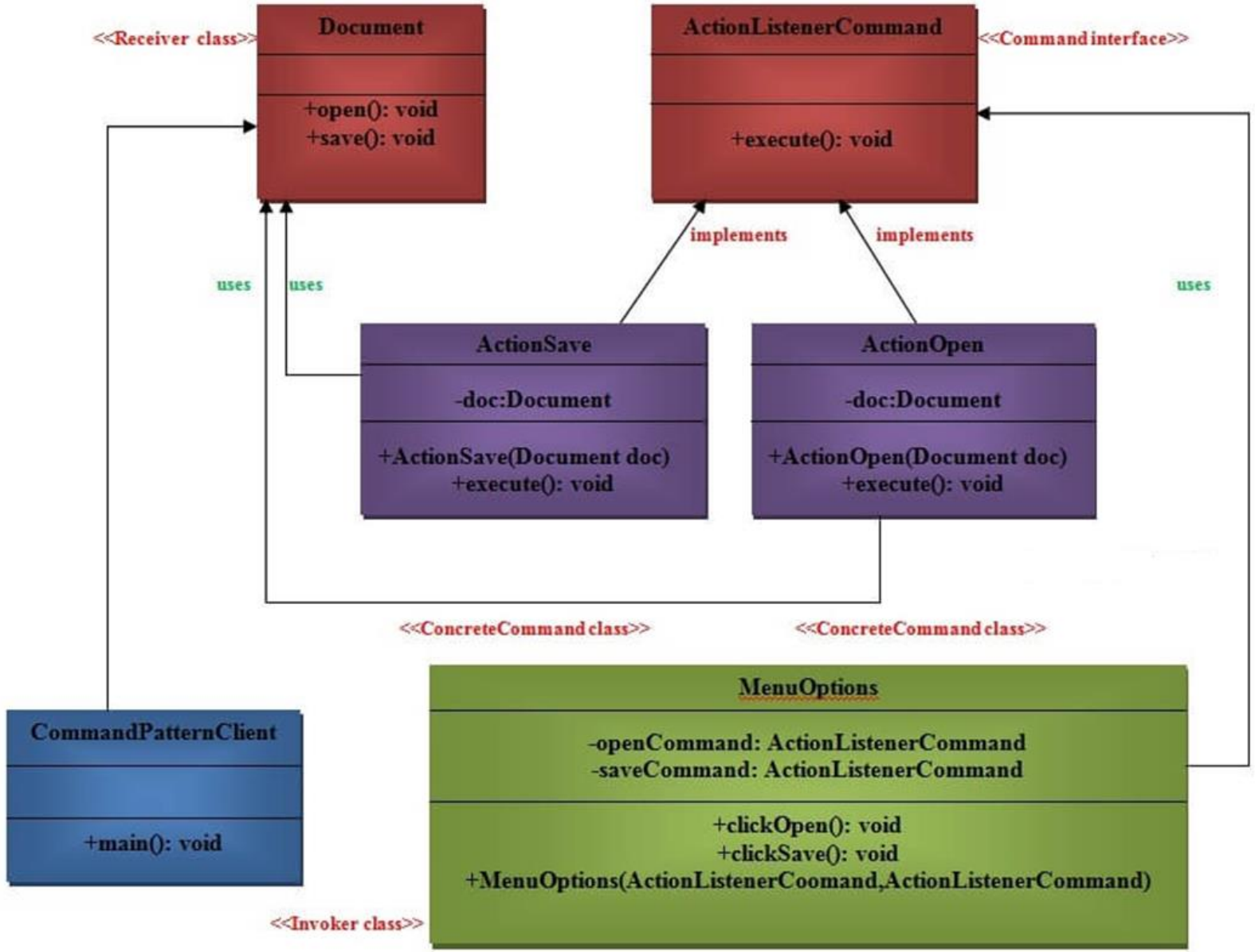
- It separates the object that invokes the operation from the object that actually performs the operation.
- It makes easy to add new commands, because existing classes remain unchanged.

Usage of command pattern:

- When you need parameterize objects according to an action perform.
- When you need to create and execute requests at different times.
- When you need to support rollback, logging or transaction functionality.

..CP

- ✓ These are the following participants of the Command Design pattern:
- **Command** This is an interface for executing an operation.
- **ConcreteCommand** This class extends the Command interface and implements the execute method. This class creates a binding between the action and the receiver.
- **Client** This class creates the **ConcreteCommand** class and associates it with the receiver.
- **Invoker** This class asks the command to carry out the request.
- **Receiver** This class knows to perform the operation.



Interpreter Pattern

- An Interpreter Pattern says that "**to define a representation of grammar of a given language, along with an interpreter that uses this representation to interpret sentences in the language".**

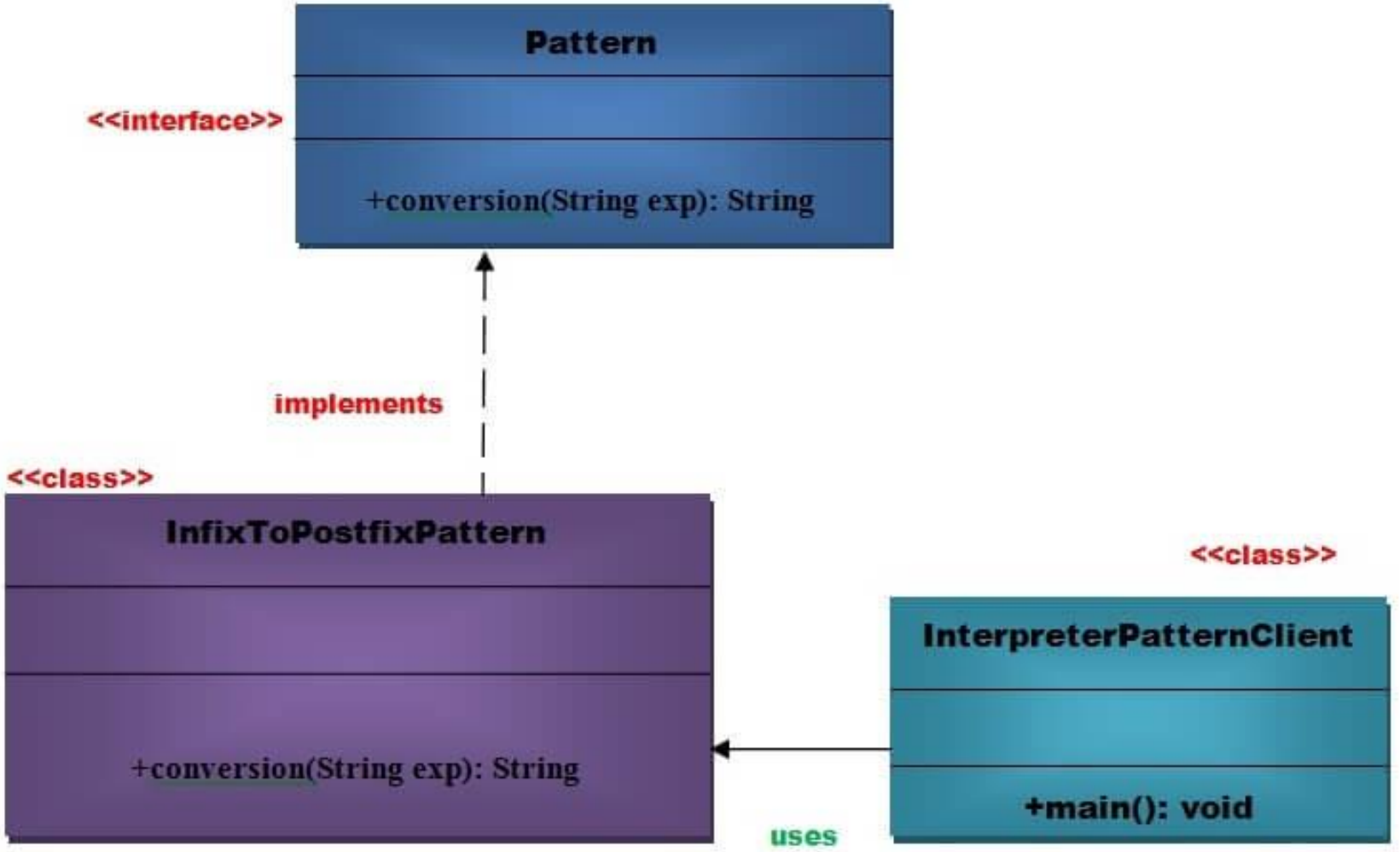
- Basically the Interpreter pattern has limited area where it can be applied. We can discuss the Interpreter pattern only in terms of formal grammars but in this area there are better solutions that is why it is not frequently used.

- This pattern can applied for parsing the expressions defined in simple grammars and sometimes in simple rule engines.

Advantage of Interpreter Pattern
- It is easier to change and extend the grammar.
- Implementing the grammar is straightforward.

Usage of Interpreter pattern:
- When the grammar of the language is not complicated.
- When the efficiency is not a priority.

....IP


Pattern
<<interface>>
+conversion(String exp): String
implements
<<class>>
InfixToPostfixPattern
+conversion(String exp): String
<<class>>
InterpreterPatternClient
+main(): void
uses

END

Next: Envisioning Architecture